MemoryLAN: A Local Area Content Replication Mesh

Christian Tschudin, University of Basel, Switzerland, Jun 18 – Jul 23, 2026

Abstract

In this note we present a distributed algorithm that maintains an approximately coherent in-network cache of memory pages. Modeled after Ethernet LANs, this "Memory LAN" provides a configuration-less and client-agnostic broadcast service suitable for high-loss wireless mesh networks. The distributed cache is filled and kept coherent on a best-effort basis using a "fast push/slow repair" approach.

The main use case for a MemoryLAN is the synchronization of replicated data types (e.g., CRDTs) within a geographically confined region. Simulations show a two-fold gain when compared to plain store-and-forward meshes a) due to broadcasting and b) due to local repair of the in-network cache. First real-world deployments using LoRa and Long Range BLE seem to confirm these gains.

1 Overview

Consider a localized distributed system whose end nodes (peers) store large sets of memory pages that they wish to replicate, either fully or partially. We interconnect these end nodes via a "Memory LAN" that provides a caching service for a small amount of memory pages. The goal is to geographically extend a physical broadcast domain (e.g., Bluetooth Low Energy or LoRa) to a mesh network with multi-hop content replication, in a transparent way.

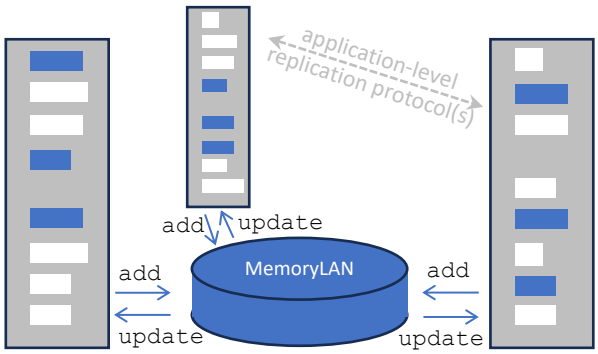


Figure 1: End nodes interconnected via a MemoryLAN: The `add` event injects new memory content, `update` delivers one or more copies of the new content to all attached end nodes.

To this end we define special store-and-forward elements that together form the MemoryLAN. The profile of these forwarders is that they are equipped with a limited amount of memory and have limited computing capacity. We call such store-and-forwarding elements "memory switches" in order to point out their similarity with Ethernet switches. Ethernet switches extend the concept of a shared physical bus to an arbitrarily interconnected set of buses, yet presenting this Local Area Network to end nodes as if it still was a single bus. In the same way we wish to extend a shared memory cache from direct neighbors to all end nodes attached to the MemoryLAN. Unlike the communication-oriented Ethernet service semantics (packet-delivery), we adopt a memory view: The effect of an `add` operation at one end node is that all peers will receive one or more corresponding `update` events, on a best-effort basis. In the folllowing writeup we will therefore replace *datagram* with the term *memory page* as the base unit of interaction.

There are two major differences between a classic EthernetLAN switch and our MemoryLAN switch. First, our communication abstraction has no notion of sender and receiver addresses and instead serves plain memory pages. Secondly, our switches can locally repair loss of cache updates occuring inside the MemoryLAN due to their their copy of the shared cache. A third nuance is the potential extension to delay-tolerant broadcast as well as the use of advanced encoding techniques like rateless fountain codes and network coding (not discussed in this note).

Overall, our approach copies the highly successful Ethernet LAN communication model of serving a geographic area larger than immediate physical neighborhood but "operating below the routing threshold". The Memory LAN answers to the need for replicating the content of end systems that are geographically close and have a common interest in shared content. This covers use cases like researchers at the same institute working with large data pools and research literature, as well as the shared photo album in a family and friends context, alike.

In our first prototype implementation, we could insert a wireless multi-hop MemoryLAN between tinySSB end nodes without having to change the actual CRDT synchronization protocol.

2 A Memory Switch Protocol

A MemoryLAN consists of interconnected memory switches that aim at maintaining LAN-wide coherent cache content. The protocol among these switches is fully symmetric, as shown in Algorithm 1, and is based on the following two techniques: **fast push with loop prevention**, and **cache assimilation with mitigation of NACK implosion** (slow repair).

In a nutshell, the system works by flooding new memory pages. Loops are prevented by a fixed-size FIFO blacklist (history) of recently forwarded packets. In parallel, new memory pages enter at the head of a fixed-size FIFO content queue (=cache) and old cache entries are automatically evicted in that process. A fingerprint (Bloom filter) of a switch's cache is periodically broadcast such that neighbors can detect cache discrepancies. Neighbors send curtesy copies of missing content, modulated by node-density in order to prevent a NACK implosion. The reception of a memory page that is already present in the cache leads to moving that page to the front of the queue i.e., to "keep this content hot" and to let the distributed cache converge towards a coherent subset of shared content.

2.1 Fast Push (Flooding)

Fast Push is the normal dissemination path for novel content: A new memory page received either from an attached end node (line 4 in Algorithm 1), or received from a neighbor memory switch (line 6), is flooded on all interfaces *after* some internal cache update and filtering that we describe now.

If it's new content, it is added to the cache. Otherwise the respective cache line is moved to the head of the cache FIFO. In this way we nudge all "hot content which is talked about a lot" away from the FIFO's tail where cold content sits and risks to be evicted.

The hash value of the incoming memory page is then checked against a history blacklist: Forwarding is supressed in case of a positive match (lines 13 and 14). Otherwise the switch adds this hash value to the blacklist and broadcasts the memory page on all interfaces, as well as delivering it to all attached end nodes (`update` event, line 21). Similar to above, the "coldest hash" is removed from the blacklist if it would become too large (lines 11 and 12).

Algorithm 1 Memory Switch (having unique $myID$)

```
1:  C ← FIFO queue                                   ▷ content cache
2:  H ← FIFO queue                                   ▷ hash history
3:  d ← 0                          ▷ neighbor density estimate, per interface

4:  on add(m):                                ▷ novelty from an attached end node
5:      incomingMemoryPage(m)

6:  on incomingMemoryPage(m):                 ▷ from network or end node
7:      if m ∈ C then
8:          move m to the head position of C
9:      else
10:         C.push(m)                                    ▷ insert at the head
11:         if |C| > CACHELIM then
12:             C.dequeue()                              ▷ drop tail element
13:     h ← hash(m)
14:     if h ∈ H then return                  ▷ do not flood duplicates
15:     H.push(h)                                        ▷ insert at the head
16:     if |H| > HISTLIM then
17:         H.dequeue()                                  ▷ drop tail element
18:     forall interfaces i:                             ▷ flooding
19:         i.broadcast(m)
20:     forall attachedEndNodes e:                       ▷ delivery
21:         e.update(m)

22: periodically :                                ▷ (request slow repair)
23:     b ← mk_bloom({hash(c) : c ∈ C})
24:     forall interfaces i:
25:         i.broadcast(BF(myID, b))

26: on incoming(BF(id, b)):                   ▷ (answer a repair request)
27:     using id and incoming interface, update d
28:     M ← {m ∈ C : hash(m) ∉ b}                ▷ M has cache deltas
29:     for most recent m ∈ M do                 ▷ pick one cache delta
30:         forall interfaces i:
31:             i.broadcast(m)                   ▷ reply with some probability
32:                                              ▷ depending on per-interface d
```

2.2 Slow Repair (Cache Assimilation)

Memory switches attempt to assimilate their direct neighbor's cache content by beaconing a Bloom filter computed over their own cache content (lines 22 to 25). Neighbors can then positively identify memory pages that the beaconing node is missing (although this is no guarantee that all differences are discovered, wherefore we randomize the Bloom filter with each beacon message).

A beacon message risks to lead to a so called NACK implosion known from multicast trees where too many nodes react on the same observation, potentially bring down the network. We prevent this by letting neighbors reply to a discovered deficit only with some probability that depends on the perceived neighborhood density. As a first approximation, a reply probability of $1/d$ is helpful where d is the number of neighbors. In practice, a more agressive dampening ($1/(2 * d)$ or $1/d^2$) is more efficient. Deriving the optimal reply probability from the number of nodes is subject to further studies.

2.3 Cache and History Eviction

Both the content cache as well as the history blacklist are a FIFO queue of fixed size. An incoming novel value (either memory page, or hash value) will simply push out the tail element from a full queue, which is an effective strategy to identify and evict stale information.

Other techniques, for example using absolute age of an entry, could also be envisaged, but lead to more complex parametrization question, we think. For example, the size of the history blacklist should be chosen such that we can filter out "looping content". In case that there is no novelty entering the MemoryLAN, the worst case blacklist size would be the sum of all the content cache sizes (for the blacklist to capture everything). But because the memory switches attempt to converge on their cache content and have overlapping content, the blacklist can be much smaller in practice.

The opposit case of a deluge of novel memory pages also helps to keep the blacklist small: New content creates a "purge pressure" for old blacklist and cache entries, squeezing out old packets that could be the source of loops. In practice we dimensioned the blacklist to have twice the length of the content cache.

2.4 Sloshing

We carefully phrased the cache coherence property saying that memory switches attempt to converge *on a subset of their cache content*. It is easy to come up with a three-nodes-in-a-string scenario where the two border switches constantly "fix" the middle node's deficit, leading to an oscillation. We call this "sloshing" where competing waves run slowly through the MemoryLAN trying to tilt the coherence in one or the other direction.

Again, we refer to a scenario where there is sufficient "novelty pressure" from the end nodes that will marginalize or completely avoid sloshing. In all other cases we accept that the network can not converge on all slots in their caches.

Above assumption about sufficient "novelty pressure" is justified because end nodes will run their higher-level synchronization protocol which requires, on top of fast push, to repeatedly probe whether new information has been missed (remember that the MemoryLAN provides an unreliable broadcast service). In other words, full replication *might* have a steady state (all end nodes are synced) but even then, there *will* be a constant inflow of "novel-looking" probing content. This is further supported by the assumption that the memory amount to be synced by end nodes (and the description of that memory amount) is larger than the cache size of the MemoryLAN.

3 Related Work

We defer the discussion of related work and a bibliography to a full version of this writeup. The entries will include: Unslotted ALOHA, Multicast and NACK implosion, Ethernet's flood-and-learn algorithm, deforestation instead of spanning tree, cache coherence in multicore CPUs, CRDT synchronization protocols, push and pull in content-centric networks, publish/subscribe systems.